

Podstawowe typy zmiennych w języku C++

Każda wartość musi być zapisana w zmiennej określonego typu. Typ określa zakres maksymalny (ile bajtów pamięci musimy zarezerwować). Typ określa interpretację danych (z czym mamy do czynienia).

Nie trzeba znać na pamięć wszystkich zakresów dokładnie, zwłaszcza tych dużych liczb. Wystarczy wiedzieć „mniej więcej” jakie są zakresy, aby wiedzieć, gdzie jest „około 65 tysięcy”, a gdzie „około 4 miliardy”.
Rozmiary w bajtach już by znać wypadało. :)

Podstawowe typy zmiennych:

Liczby całkowite ze znakiem (dodatnie i ujemne)

`char` - rozmiar: 1 bajt - zakres: od -128 do 127 `short` -
rozmiar: 2 bajty - zakres: od -32'768 do 32'767 `int` -
rozmiar: 4 bajty - zakres: od -2'147'483'648 do
2'147'483'647

`long` - rozmiar: 4 bajty - zakres: od -2'147'483'648 do
2'147'483'647

`long long` - rozmiar: 8 bajtów - zakres: od
-9'223'372'036'854'775'808 do
9'223'372'036'854'775'807

Uwagi:

- Do powyższych zmiennych można dopisać typ z przodu **signed** (np. **signed int**), aby wskazać, że mamy typ ze znakiem. Dla wygody programisty jednak pomija się ten zapis. Zakłada się po prostu, że domyślnie wartości są ze znakiem, jeśli nie mają tego dopisku.
- Najczęściej stosowanym typem jest **int**. Jego rozmiar jest dostosowany do domyślnego rozmiaru słowa procesora, przez co (teoretycznie) operacje na tym typie są wykonywane najszybciej (typy „krótkie” (**char**, **short**), będą najpierw konwertowane na typ **int**, później przeprowadzane będą obliczenia, następnie konwertowane ponownie na oryginalny typ, a to zajmuje czas procesora. Typy „dłuższe” (**long long**) mogą być rozbijane na dwie mniejsze części (po 4 bajty), operacje przeprowadzone oddzielnie, na końcu wynik „sklejany” ponownie w jedną większą).
- Typ **int** może mieć różny rozmiar (i zakres) w zależności od kompilatora i ustawień kompilacji. Dawno, dawno temu typ ten miał 2 bajty (tyle samo co `short`), w 64-bitowej kompilacji może (ale nie musi) mieć 8-bajtów (tyle samo co `long long`).

Generalnie jak używamy `int`'a, nie przejmujemy się tym zjawiskiem. :) •

Typ **char** ma podwójne zastosowanie. Po pierwsze przechowuje liczby z małego zakresu. Po drugie, jego domyślna interpretacja to znak (w postaci kodu ASCII z tabeli). Także jeśli zmienna `char` ma wartość 65, to wyświetlając ją na ekranie nie zobaczymy liczby 65 tylko literę 'A'.

(Program pomyśli, że skoro drukujemy typ **char**, to pewnie chcemy wyświetlić znak z tablicy ASCII o numerze 65).

Podstawowe typy zmiennych w języku C++

Liczby całkowite bez znaku (tylko dodatnie)

unsigned char – rozmiar: 1 bajt – zakres: od 0 do 255
unsigned short – rozmiar: 2 bajty – zakres: od 0 do 65'535
unsigned int – rozmiar: 4 bajty – zakres: od 0 do 4'294'967'295
unsigned long – rozmiar: 4 bajty – zakres: od 0 do 4'294'967'295
unsigned long long – rozmiar: 8 bajtów – zakres: od 0 do 18'446'744'073'709'551'615

Uwagi:

- Typy takie same jak poprzednio. Z tym, że z dopisanym wcześniej **unsigned**. Deklaracja wymusza interpretację pierwszego bitu nie jako znak, ale jako wartość (tracimy liczby ujemne, ale 2 razy zwiększamy zakres „na plusie”).
- Deklaracja samego **unsigned**, ma takie samo znaczenie jak deklaracja **unsigned int** (chodzi tylko o wygodę programisty).

Liczby z przecinkiem:

float – rozmiar: 4 bajty – dokładność: około 6 cyfr znaczących
double – rozmiar: 8 bajtów – dokładność: około 15 cyfr znaczących
long double – rozmiar: (może być 10, 12, 16 bajtów), zależy od kompilatora i architektury procesora

Uwagi:

- Typ zmiennoprzecinkowy jest typem ze znakiem.
- Typ zmiennoprzecinkowy przechowuje wartości z określoną dokładnością. Liczbę np. 2 uda mu się zapisać dokładnie. Liczbę np. 0.1 już nie da się zapisać idealnie. Należy o tym pamiętać, w trakcie wykonywania obliczeń bardzo zaawansowanych i super precyzyjnych. *Coś co matematycznie wychodzi, że jest równe, może nie być równe w programowaniu.*

Typ logiczny:

bool – rozmiar: 1 bajt – tylko 2 stany prawda lub fałsz

Uwagi:

- Ze względów technicznych nie można zapisać tej zmiennej na jednym bicie (choć tylko 2 stany tak sugerują). Chodzi o sposób adresacji pamięci, gdzie procesor naraz „połyka” po 8 bitów. Coś za coś. Tracimy trochę bitów, ale za to mamy łatwy dostęp do pamięci.
- Technicznie zmienna ta traktuje wszystkie 8 bitów ustawione na 0 jako fałsz. Jeśli jakiś z tych bitów będzie ustawiony na 1, zmienna będzie potraktowana jako prawda.
- Co prawda wyświetlenie wartości typu **bool** wyświetli na ekranie 0 lub 1, ale (profesjonalnie) przypisywać powinniśmy **true** albo **false**.

Typ znakowy:

char[] – tablica znaków. Kolejne elementy przechowują kolejne kody znaków ASCII.

Wymagany znany rozmiar, bo musimy wiedzieć ile potrzebujemy zarezerwować miejsca w pamięci. Szczegółowy opis w dalszym rozdziale.

string – inteligentny łańcuch znaków, sam się rozciąga i kurczy w razie potrzeby.

Szczegółowy opis w dalszym rozdziale.

Podstawowe operacje na zmiennych liczbowych

```
int a = 13;
int b = 5;
int c;

c = a + b; // Suma
c = a - b; // Różnica
c = a * b; // Mnożenie

// Dzielenie. Uwaga! Należy zawsze patrzeć na typy
// zmiennych. // Po pierwsze, zmienna przechowująca wynik
// powinna być typu // zmiennoprzecinkowego (float, double).
// Inaczej wynik będzie // całkowity.
// Po drugie, w przypadku, gdy obie liczby (dzielna i
dzielnik) // będą typu całkowitego, to wynik także będzie miał
wartość // całkowitą (nawet, gdy zmienna przechowująca wynik
// będzie // typem zmiennoprzecinkowym).
// W przykładzie wynikiem będzie 2.
c = a / b;

// Operator dzielenia z resztą, tzw. modulo.
// Zwraca resztę z dzielenia liczby przez
// liczbę. // Działa tylko dla typów całkowitych.
// W przykładzie wynikiem będzie 3.
c = a % b;
```

Skrócony zapis operacji.

Poniższe zapisy mają to samo działanie.

```
x = x + 1; // Do zmiennej x dodaj 1, wynik zapisz w zmiennej x.

x += 1; // Dodaj liczbę 1 do zmiennej x. Działa dla pozostałych
// operacji, tj.: +=, -=, *=, /=, %=, itp..

x++; // Zwiększ x o jeden (inkrementacja). Działa także dla
// zmniejszania o 1 (dekrementacji), tj.: x--;
```

Operacje na zmiennych

Procesor wykonuje operacje kolejno zgodnie z zasadami kolejności działań. (*,/,%) przed (+,-), itp.. W miarę wykonywania kolejnych operacji, o ile będzie to wymagane, następuje konwersja typów, aby dało się na nich wykonać operacje.

W wielkim uproszczeniu. Procesor wykonuje jedną operację na maksymalnie dwóch zmiennych, które są tego samego typu. Jeśli zmienne są różnych typów, to nie da się wykonać na nich tej operacji. Wtedy procesor automatycznie skonwertuje jeden z typów na drugi (tzw. konwersja niejawna), tak aby ograniczyć ewentualną stratę informacji (tj. gdy dodajemy typ

int do typu **float**, to procesor zamieni pierwszy **int** na **float**, a nie odwrotnie). Konwertowanie typów zadziała nawet wtedy, gdy wykonujemy operacje na pozornie tych samych rodzajach zmiennych, np. dodajemy do siebie liczbę całkowitą typu **short** (2 bajty) oraz liczbę całkowitą typu **int** (4 bajty). Procesor najpierw skonwertuje pierwszego **shorta** do **inta**, dopiero wtedy liczbę po konwersji doda do drugiej zmiennej (**int**). Tak samo dla **float** i **double**, czy dla **int** oraz **unsigned**.

```
int a = 14;
int b = 5;
double f = 2.5;
int c = ( ( a / b ) + ( a % b ) ) * f;
```

Obliczamy:

```
c = ( ( 14 / 5 ) + ( 14 % 5 ) ) * 2.5
// Dzielenie liczb typu int, więc wynik też będzie typu int.
// Dzielenie modulo dwóch int'ów, więc wynik też będzie typu
int. c = ( 2 + 4 ) * 2.5 // Dodanie do siebie dwóch liczb typu
int.
```

```
c = 6 * 2.5
// Mnożenie przez siebie liczby typu int oraz double. Nie można
wykonać. // Musi nastąpić niejawną konwersja.
// Zmienna typu int jest automatycznie skonwertowana na typ double. c =
6.0 * 2.5 // Już są dwa te same typy, więc można wykonać to mnożenie.

c = 15.0 // Zmienna c jest typu int, także wynik musi być zamieniony z typu
//double na typ int właśnie.

c = 15 // Ostateczny wynik (typ int).
//(gdyby tam było coś po przecinku, to by uległo utraceniu).
```

Wymuszenie rzutowania, wymuszenie interpretacji wyświetlania. Dopisując przed zmienną typ w nawiasie (w tym momencie) konwertujemy wartość zmiennej na ten typ.

Uwaga! Rzutując liczbę zmiennoprzecinkową na typ całkowity **nie zaokrąglamy** liczby. Bierzemy jej całkowitą część, całkowicie ignorując część po przecinku.

```
int a = 9.999;
cout << a << endl; // Zobaczymy na ekranie wartość 9.
```

Inny przykład:

```
float X = 65.8;
cout << X << endl; // Zobaczymy na ekranie wartość 65.8 cout << (int)X << endl;
// Zobaczymy na ekranie liczbę 65, konwersja na liczbę // całkowitą (pomijamy
całkowicie część po przecinku). cout << (char)X << endl; // Zobaczymy na
ekranie literę "A", interpretacja jako // typ znakowy char.
```